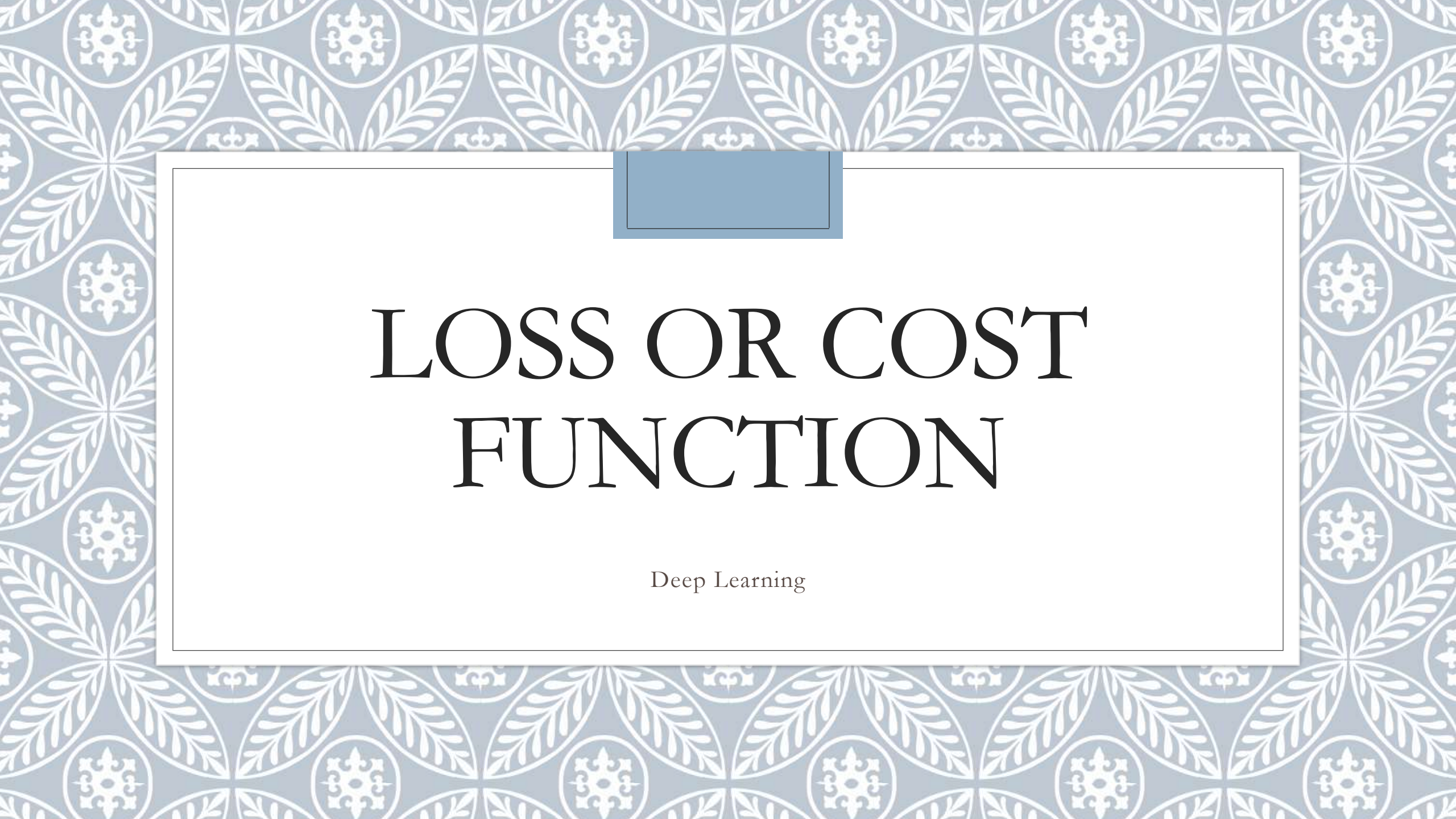


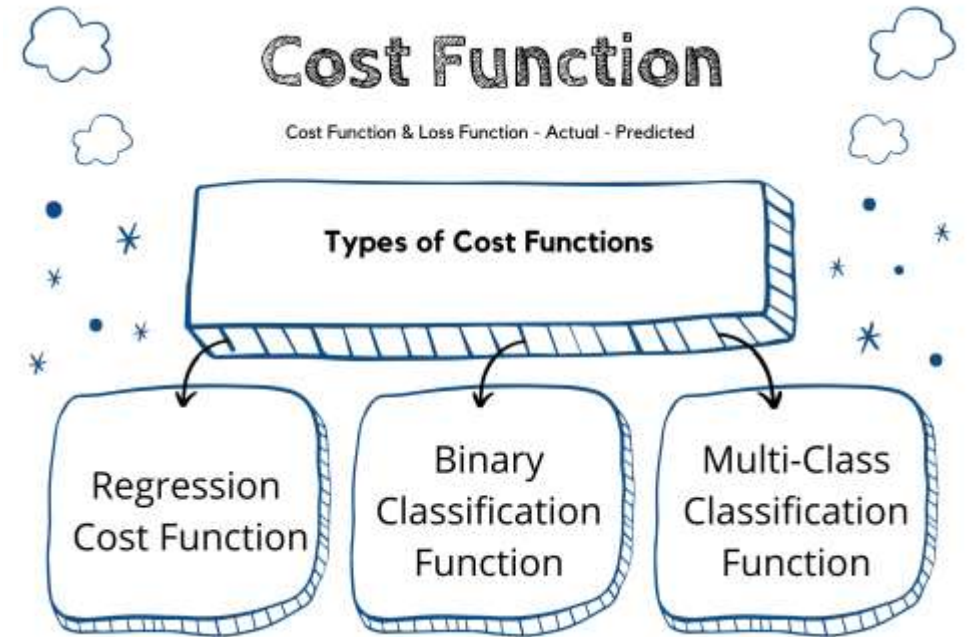


LOSS OR COST FUNCTION

Deep Learning

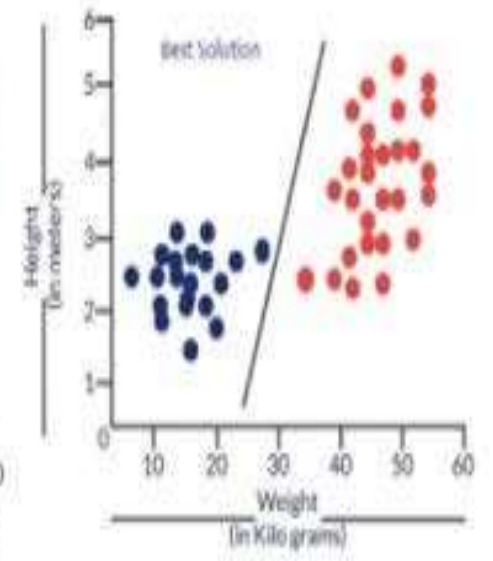
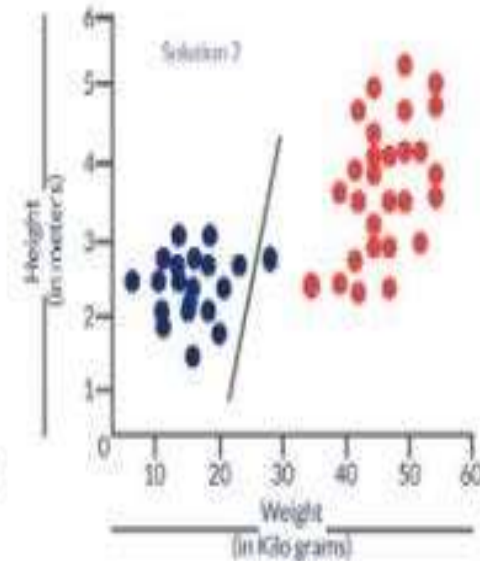
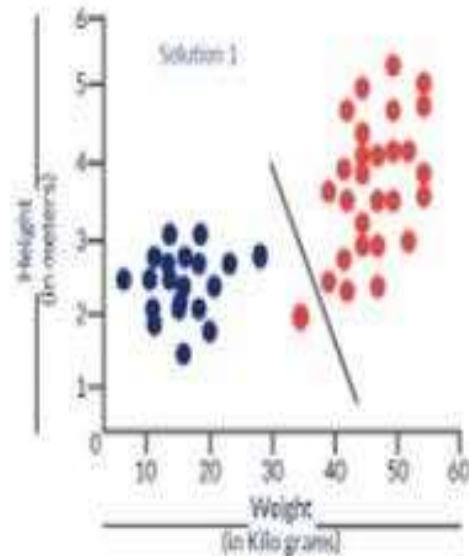
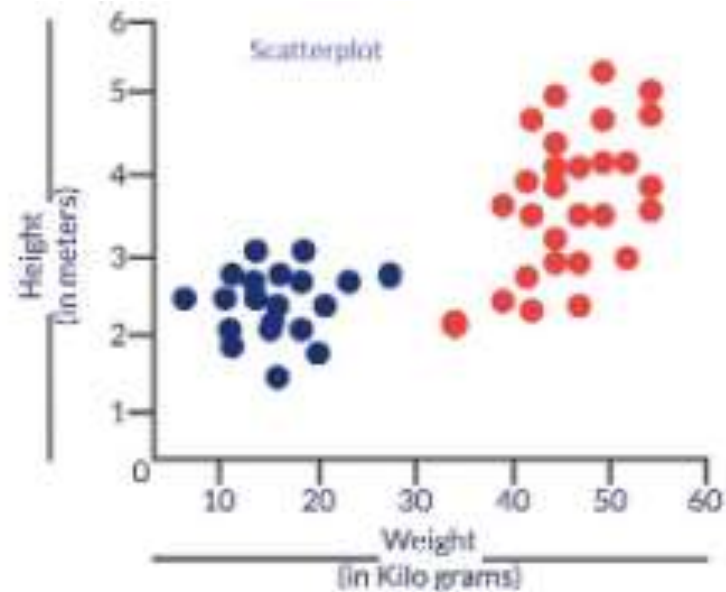
Loss/Cost Function

- ‘Loss’ in Machine learning helps us understand the difference between the predicted value & the actual value.
- The Function used to quantify this loss during the training phase in the form of a single real number is known as the “Loss Function”.
- Loss function: Used when we refer to the error for a single training example.
- Cost function: Used to refer to an average of the loss functions over an entire training data.



Why Cost Function?

- Consider a scenario where we wish to classify data. Suppose we have the height & weight details of some cats & dogs.



Regression Loss Functions:

1. Mean Squared Error/Squared loss/ L2 is always differentiable.

loss

- The difference between the actual value and the model prediction, square it, and then average it across the entire dataset.

$$\text{MSE} = \frac{1}{N} \sum_i (Y_i - \hat{Y}_i)^2$$

Advantage

- **Easy Interpretation:** The MSE is straightforward to understand.
- **Always Differential:** Due to the squaring, it

- **Single Local Minimum:** It has only one local minimum.

Disadvantage

- **Error Unit in Squared Form:** The error is measured in squared units, which might not be intuitively interpretable.
- **Not Robust to Outliers:** MSE is sensitive to outliers.

- *Note: In regression tasks, at the last neuron, it's common to use a linear activation function.*

Regression Loss Functions:

2. Mean Absolute Error/ L1 loss Functions

- It calculates the average absolute difference between the actual value and the model prediction across the dataset.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |Y_i - \hat{Y}_i|$$

Advantage

- **Intuitive and Easy:** MAE is easy to grasp.
- **Error Unit Matches Output Column:** The

- **Robust to Outliers:** MAE is less affected by outliers.

Disadvantage

- **Graph Not Differential:** The MAE graph is not differentiable, so gradient descent cannot be applied directly. Subgradient calculation is an alternative.
- *Note: In regression tasks, at the last neuron, it's common to use a linear activation function.*

Regression Loss Functions:

3. Huber Loss/Smooth Mean Absolute Error

Error

- $\hat{y}$: The predicted value returned by the model.
- The Huber loss is used in robust regression and is less sensitive to outliers compared to squared error loss.
- δ : Defines the point where the Huber loss transitions from quadratic to linear.

$$Huber = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y_i - \hat{y}_i)^2 \quad |y_i - \hat{y}_i| \leq \delta$$

$$Huber = \frac{1}{n} \sum_{i=1}^n \delta \left(|y_i - \hat{y}_i| - \frac{1}{2} \delta \right) \quad |y_i - \hat{y}_i| > \delta$$

- n : The number of data points.
- y : The actual value (true value) of the

Regression Loss Functions:

3. Huber Loss

Advantage

- **Robust to Outliers:** Huber loss is more robust to outliers.
- **Balances MAE and MSE:** It lies between MAE and MSE.

Disadvantage

- **Complexity:** Optimizing the hyperparameter δ increases training requirements.
- The Huber loss is used in robust regression and is less sensitive to outliers compared to squared error loss.

Classification Loss Functions:

1. Binary Cross Entropy/log loss Functions in machine learning models

- It is used in binary classification problems like two classes. example a person has covid or not or my article gets popular or not.
- Binary cross entropy compares each of the predicted probabilities to the actual class output which can be either 0 or 1. It then calculates the score that penalizes the probabilities based on the distance from the expected value. That means how close or far from the actual value.

$$\text{Log Loss} = -\frac{1}{N} \sum_{i=1}^N y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)$$

- y_i – actual values
- $\hat{y}$ – Neural Network prediction

Classification Loss Functions:

2. Categorical cross-entropy

- It is a loss function that is used for single-label categorization. This is when only one category is applicable for each data point. In other words, an example can belong to one class only.

$$L(y, \hat{y}) = - \sum_{j=0}^M \sum_{i=0}^N (y_{ij} * \log(\hat{y}_{ij}))$$

- Categorical cross-entropy will compare the distribution of the predictions (the activations in the output layer, one for each class) with the true distribution, where the probability of the true class is set to 1 and 0 for the other classes.

Multi-class Classification Loss Functions

1. Multi-class Cross Entropy Loss:

- A multi-class classification cost function is used in the classification problems for which instances are allocated to one of more than two classes.
- Here also, similar to binary class classification cost function, cross-entropy or categorical cross-entropy is a commonly used cost function.

$$-\sum_{c=1}^M y_{o,c} \log(p_{o,c})$$

M — No of classes

y — binary indicator (0 or 1) if class label *c* is the correct classification for input *o*

p — predicted probability score of input *o* belonging to class *c*.

Multi-class Classification Loss Functions

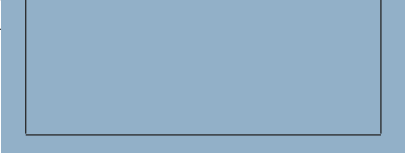
1. Multi-class Cross Entropy Loss:

Advantages:

- **Handles Imbalanced Data:** By adjusting the predicted probabilities, it can handle imbalanced datasets better than other loss functions like mean squared error
- **Flexibility:** Can be easily extended to handle different types of classification problems, including binary, multi-class, and even multi-label classification with slight modifications.

Disadvantages:

- **Sensitive to Mislabeling:** Sensitive to mislabeled data, which might heavily penalize the model for incorrect predictions, leading to poor performance if the dataset has a significant number of mislabeled instances.



THANK YOU!!